

● LangGraph · LangChain · Agentic RAG · ChromaDB

PsycheNavigator

Technical Deep-Dive

A complete explanation of the LangGraph agent, agentic RAG pipeline, vector database, embeddings, and all 8 processing nodes.

8

Agent Nodes

5,640

Book Chunks

7

Psychology Books

384

Embedding Dims

PsycheNavigator — Full Technical Explanation

Table of Contents

1. What is this project?
 2. LangChain — The Toolkit
 3. LangGraph — The Brain
 4. All 8 Agent Nodes — Explained
 5. Agentic RAG — What Makes It "Agentic"
 6. Vector Database (ChromaDB)
 7. Embeddings — Turning Text into Numbers
 8. Chunks — Splitting the Books
 9. The Full Pipeline — Step by Step
 10. The Hybrid Retrieval System
-

1. What is this project?

PsycheNavigator is a psychology assistant that answers personal questions (conflicts with managers, relationships, self-doubt, etc.) by reading real psychology books and generating evidence-based advice.

The user types a problem → the system reads relevant book excerpts → an AI generates a warm, cited response → an action plan is created.

The key difference from a simple chatbot: **it does not rely on the AI's memory**. Every response is grounded exclusively in text retrieved from 7 psychology books stored in a vector database.

```
User types a problem
    ↓
LangGraph runs 8 nodes in sequence
    ↓
Reads real book excerpts (via Hybrid RAG)
    ↓
Generates response + action plan
    ↓
Streams everything live to the frontend
```

2. LangChain — The Toolkit

LangChain is NOT the AI itself. It is a **toolkit** (a collection of ready-made building blocks) that connects AI models, databases, and tools together.

Think of it as LEGO pieces that snap together. Without LangChain, you would need to write custom code to connect each piece manually.

What LangChain provides in this project:

A) The Pipe Operator |

```
response = (prompt | llm).invoke({"user_message": "I struggle with my boss"})
```

This chains: format the prompt → send to AI → get response. Without LangChain, this would be 15+ lines of manual API calls and string formatting.

B) ChatPromptTemplate — Prompt Builder

```
prompt = ChatPromptTemplate.from_template("""
Classify the situation: {user_message}
Return only: manager / colleague / partner / family / self
""")
```

LangChain handles injecting variables and formatting the system/human message structure that AI APIs require. Every node in this project uses a prompt template.

C) The Entire Retrieval Stack

LangChain wraps every retrieval component with the same `.invoke()` interface:

Component	What it does	LangChain class
ChromaDB	Vector search	<code>Chroma</code>
Sentence Transformers	Embed text	<code>HuggingFaceEmbeddings</code>
BM25	Keyword search	<code>BM25Retriever</code>
Ensemble	Combine BM25 + dense	<code>EnsembleRetriever</code>
Cohere Rerank	Neural reranking	<code>CohereRerank</code>
Full pipeline	All together	<code>ContextualCompressionRetriever</code>

Because they all share the same interface, you can swap any piece without rewriting the rest.

D) PydanticOutputParser — Structured Output

```
class ActionPlan(BaseModel):
    steps: List[str]
    framework_used: str
    book_source: str
    time_horizon: str

parser = PydanticOutputParser(pydantic_object=ActionPlan)
result = (prompt | llm | parser).invoke({...})
result.steps  # already a Python list, not raw text
```

LangChain tells the AI "respond in this exact JSON format" and automatically parses the raw text response into a Python object. Without this, JSON parsing would need manual error handling.

3. LangGraph — The Brain

LangGraph is a framework for building **stateful AI agents** — programs that remember what happened in previous steps and make decisions based on that history.

The Key Concept: AgentState

Every node reads from and writes to a shared state dictionary:

```
class AgentState(TypedDict):  
    messages: List[dict] # full conversation history  
    situation_type: str # "manager" / "colleague" / etc.  
    current_query: str # what to search for  
    retrieved_chunks: List[dict] # book excerpts found  
    retrieval_attempts: int # how many times we tried (max 3)  
    is_enough: bool # did evaluator approve the chunks?  
    retry_reason: str # why evaluator said insufficient  
    active_frameworks: List[str] # which books were used  
    action_plan: List[str] # structured action steps  
    final_response: str # the AI's final answer  
    should_loop: bool # loop back for follow-up?  
    turn_count: int # conversation turn counter  
    is_off_topic: bool # did guard block this?
```

This state is passed from node to node. Each node reads what it needs and writes its output back. This is how nodes "talk" to each other.

The Graph Structure



LangGraph compiles this graph at server startup with `graph.compile()`. This converts it into an optimized executable. Every time a user sends a message, the graph runs from the entry point (Classifier) to END.

4. All 8 Agent Nodes — Explained

Node 1: Classifier

File: `backend/agent/nodes/classifier.py`

What it does: Reads the user's message and decides what type of situation this is. This is important because different situations should search different books (e.g., manager conflicts → Crucial Conversations, partner issues → Attached).

How it works: - Sends the message to Groq (Llama 3.3 70B) with temperature=0 (deterministic) - The AI must respond with exactly one word: manager / colleague / partner / family / self / off_topic - If off_topic: sets `is_off_topic = True` → routes to Guard - If valid: sets `situation_type` → routes to Retriever

Why temperature=0: Classification needs consistency. The same input should always produce the same output. Temperature=0 removes all randomness.

```
# Input: "my manager never listens to me"
# Output: situation_type = "manager"

# Input: "what is the weather today?"
# Output: is_off_topic = True
```

Node 2: Guard

File: `backend/agent/nodes/guard.py`

What it does: Short-circuits the entire pipeline for off-topic questions. Instead of wasting 5 LLM calls on "who won the football match?", Guard immediately returns a polite refusal.

Why this matters: Without Guard, every message goes through the full pipeline. A question about football would: run retrieval on psychology books, evaluate psychology chunks against a football question (INSUFFICIENT), retry 3 times, and finally produce a confused answer. Total: 6+ LLM calls wasted. With Guard: 1 LLM call (Classifier), instant response.

```
Off-topic path: Classifier → Guard → END (1 LLM call, ~1 second)
Normal path:   Classifier → Retriever → ... → END (5+ LLM calls, ~5
seconds)
```

Node 3: Retriever

File: `backend/agent/nodes/retriever.py`

What it does: Searches the knowledge base of 5,640 book chunks to find the most relevant passages for the user's question.

How it works: 1. Looks up which books are relevant to the `situation_type`: `python`

```
SITUATION_BOOK_MAP = { "manager": ["Crucial Conversations", "Never Split the Difference", "The 7 Habits"], "colleague": ["Crucial Conversations", "Nonviolent Communication", "The Like Switch"], "partner": ["Attached", "Nonviolent Communication", "Radical Acceptance"], "family": ["Attached", "Radical Acceptance", "Nonviolent Communication"], "self": ["Radical Acceptance", "The 7 Habits", "Attached"], }
```

2. Filters the 5,640 chunks down to ~700-900 from only the relevant books 3. Runs Hybrid Search (BM25 + Dense Vector — explained in Section 10) 4. Uses Cohere to rerank and keep only the top 4 chunks 5. Saves those 4 chunks to `retrieved_chunks` in the state 6. Increments `retrieval_attempts` by 1

Why filter by book first? If you ask about a partner conflict, searching through "Never Split the Difference" (FBI negotiation tactics) is wasteful and noisy. Pre-filtering by situation type dramatically improves relevance.

Node 4: Evaluator

File: `backend/agent/nodes/evaluator.py`

What it does: Acts as a quality gate. It reads the retrieved chunks and the user's question and asks: "Is this good enough to give real advice?"

How it works: - Sends chunks + user question to Groq (temperature=0) - The AI must respond with exactly: `SUFFICIENT` or `INSUFFICIENT` - Then one sentence explaining why

What makes chunks "sufficient": 1. Do they contain a real actionable framework? (not just vague theory) 2. Do they directly relate to this specific situation? 3. Is there enough for concrete, empathetic advice?

Routing logic:

```
def route_after_evaluation(state):
    if state["is_enough"]:
        return "psychologist"    # good chunks → proceed
    if state["retrieval_attempts"] >= 3: return "psychologist"    # max retries → force answer
    return "query_builder"      # bad chunks → refine query
```

This is the core of what makes the system "agentic" — it doesn't just blindly proceed, it checks quality and adapts.

Node 5: Query Refiner (query_builder)

File: `backend/agent/nodes/query_builder.py`

What it does: When the Evaluator says chunks are insufficient, this node rewrites the search query to find better results.

How it works: - Receives: original user message + the Evaluator's reason for rejection - Sends to Groq (temperature=0.3 — slightly creative) - The AI generates a better, more specific search query - Updates `current_query` in the state - Execution goes back to Retriever (this is the retry loop)

Example:

```
User: "I feel invisible at work"
Original query: "I feel invisible at work"
Evaluator says: "Chunks discuss general communication but not workplace
visibility"
Refined query: "being overlooked ignored at work recognition acknowledgment"
```

The refined query uses keywords more likely to match relevant book passages. Maximum 3 retries before forcing the Psychologist to answer anyway.

Node 6: Psychologist

File: `backend/agent/nodes/psychologist.py`

What it does: Generates the main response — a warm, empathetic, evidence-based answer grounded exclusively in the retrieved book excerpts.

How it works: - Builds context from all 4 retrieved chunks: `` [Crucial Conversations, p.47] "When we face crucial conversations, we often..."

[The 7 Habits, p.152] "Seek first to understand, then to be understood..." `` - Sends this context + user message + full conversation history to Groq (temperature=0.7 — warm and human-sounding, not robotic) - The system prompt instructs: - Acknowledge feelings before giving advice - Use ONLY the provided book excerpts as your source - Cite the book name when referencing a framework - Never diagnose, never prescribe, always empower - Updates `final_response` and `active_frameworks`` (which books were used)

Why temperature=0.7: The classifier needs determinism (temp=0). The psychologist needs warmth and natural language variation (temp=0.7). Higher temperature = more creative and human-sounding responses.

Node 7: Action Planner

File: `backend/agent/nodes/action_planner.py`

What it does: Converts the Psychologist's response into a concrete, structured action plan with specific steps the user can take tomorrow.

How it works: - Uses `PydanticOutputParser` with this schema: `python class ActionPlan(BaseModel): steps: List[str] # e.g. ["Schedule a 1:1 meeting", "Use 'I' statements"] framework_used: str # e.g. "Crucial Conversations framework" book_source: str # e.g. "Crucial Conversations" time_horizon: str # e.g. "This week"` - LangChain automatically tells the AI to respond in this JSON structure - LangChain automatically parses the AI's text into a Python object - Steps must be "specific, behavioural, immediately actionable. Not abstract."

Why structured output: The frontend needs to render the action plan as a numbered list with specific fields. If the AI returns free text, parsing breaks. Pydantic schema enforces the contract.

Node 8: Follow-up

File: `backend/agent/nodes/follow_up.py`

What it does: Generates one targeted follow-up question to deepen the conversation or uncover something that would change the advice.

How it works: - Receives: user's original message + AI's response + action plan - Generates ONE short question that either: - Digs deeper into something unknown ("How long has this been going on?") - Checks how they feel about the plan ("Does step 2 feel realistic for you?") - Uncovers context that would change advice ("Is your manager new to the role?") - Appends the follow-up question to `final_response` - Sets `should_loop = False` (no looping back in current implementation)

5. Agentic RAG — What Makes It "Agentic"

Regular RAG (Retrieval Augmented Generation):

```
User question → Retrieve chunks → Generate answer
```

Simple. One-shot. No checking. No adaptation.

Agentic RAG (this project):

The word "agentic" means the system makes decisions and takes actions to achieve a goal. A regular RAG doesn't check if results are good. This system does — it evaluates, reasons about quality, decides to retry with a better query, and only proceeds when satisfied (or after maximum attempts).

The **retry loop** (Evaluator → Query Refiner → Retriever → Evaluator → ...) is the key agentic behavior. The system is not just retrieving — it is actively trying to improve its own retrieval quality.

6. Vector Database (ChromaDB)

What is a Vector Database?

A normal database stores text and searches by exact keyword matching:

```
SELECT * FROM chunks WHERE text LIKE '%conflict%'
```

This misses "disagreement", "argument", "tension" — same concept, different words.

A vector database stores text as **vectors** (lists of numbers) and searches by **semantic similarity** — meaning, not exact words:

ChromaDB in this project

Storage: All 5,640 book chunks are stored as vectors in a local SQLite file (`backend/chroma_db/`). This is loaded once at server startup.

Index type: HNSW (Hierarchical Navigable Small World) - A graph-based index that makes searching millions of vectors fast - Instead of comparing your query to all 5,640 chunks one by one (slow), HNSW navigates a layered graph to find similar vectors in milliseconds

How a search works: 1. User query "I feel invisible at work" is converted to a vector [0.23, -0.15, 0.87, ...] 2. ChromaDB finds the 8 chunks whose vectors are closest to that query vector 3. "Closest" means most semantically similar in meaning

7. Embeddings — Turning Text into Numbers

The model: `sentence-transformers/all-MiniLM-L6-v2` (runs locally on CPU)

What is an embedding?

An embedding is a way to represent the MEANING of text as a point in mathematical space. Similar meanings end up close together.

```
"I hate my boss"           → [0.23, -0.15, 0.87, 0.44, ...] (384 numbers)
"conflict with manager"    → [0.21, -0.14, 0.85, 0.46, ...] (very similar!)
"I love pizza"             → [0.89, 0.67, -0.23, 0.11, ...] (very
different)
```

The 384-dimensional vector captures the full semantic meaning of the sentence.

How embeddings are created

During ingest (one-time setup):

```
PDF book
  ↓ PyPDF loads pages
  ↓ RecursiveCharacterTextSplitter creates 800-token chunks
  ↓ all-MiniLM-L6-v2 converts each chunk to a 384-dim vector
  ↓ ChromaDB stores (chunk text + vector + metadata)
```

During query (every request):

```
User query → all-MiniLM-L6-v2 → 384-dim vector → ChromaDB search
```

The same model is used for both so the query vector and chunk vectors exist in the same mathematical space and can be meaningfully compared.

Why all-MiniLM-L6-v2?

- Small and fast (runs on CPU in ~50ms)
 - 384 dimensions — good balance of quality vs speed
 - Trained specifically for semantic sentence similarity
 - Open source, runs locally, no API cost
-

8. Chunks — Splitting the Books

Why split books into chunks?

AI models have a **context limit** — they can only read a certain amount of text at once. You cannot feed an entire 300-page book to the AI. You need to find and send only the relevant pages.

Also: embedding a whole chapter produces a vague vector that represents everything and nothing specifically. Smaller chunks produce precise, focused embeddings.

How chunks are created (ingest.py)

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=800,          # each chunk is ~800 tokens (~600 words)  
    chunk_overlap=120,      # 120-token overlap between consecutive chunks  
)
```

chunk_size=800: Small enough to have a focused meaning. Large enough to contain a complete idea or framework explanation.

chunk_overlap=120: If an important sentence falls at the end of one chunk and beginning of the next, the overlap ensures it's fully captured in at least one chunk.

Example of chunking

```
Original text (simplified):

"...The key to crucial conversations is psychological safety.
When people feel safe, they can share anything. Safety is the
air that dialogue breathes. [chunk 1 ends here]

[chunk 2 starts 120 tokens before]
...breathes. Without safety, crucial conversations fail.
The two most common threats to safety are..."

Chunk 1: "...The key to crucial conversations is psychological safety.
        When people feel safe, they can share anything. Safety is the
        air that dialogue breathes."

Chunk 2: "...breathes. Without safety, crucial conversations fail.
        The two most common threats to safety are..."
```

Metadata stored with each chunk

```
{
  "text":  "The key to crucial conversations...",
  "source": "Crucial Conversations",      # book name (used for filtering)
  "page":  47,                          # page number (shown in
citations)
}
```

The knowledge base

Book	Chunks
Crucial Conversations	~900
Never Split the Difference	~750
Nonviolent Communication	~650
The 7 Habits	~850
Attached	~780
Radical Acceptance	~720
The Like Switch	~690
Total	~5,640

9. The Full Pipeline — Step by Step

Let's trace a real request: "My manager never listens to me in meetings"

STEP 1 – User sends message

```
Frontend POST /chat {"messages": [{"role": "user", "content": "My manager never listens to me in meetings"}]}
```

STEP 2 – Classifier Node (LLM call #1)

Input: "My manager never listens to me in meetings"

LLM: Groq Llama 3.3 70B, temperature=0

Output: "manager"

State: situation_type = "manager"

Stream: {"node": "classifier", "output": {"situation_type": "manager"}}

STEP 3 – Retriever Node (no LLM, but Cohere API call)

Filter books: ["Crucial Conversations", "Never Split the Difference", "The 7 Habits"]

BM25 search: "My manager never listens to me in meetings" → 8 keyword matches

Dense search: embed query → find 8 semantically similar vectors in ChromaDB

Ensemble: merge 16 results (BM25 weight 0.4, Dense weight 0.6)

Cohere: rerank all 16 → keep top 4

```
State: retrieved_chunks = [
    {text: "...", source: "Crucial Conversations", page: 47},
    {text: "...", source: "The 7 Habits", page: 152},
    {text: "...", source: "Never Split...", page: 89},
    {text: "...", source: "Crucial Conversations", page: 78},
]
```

retrieval_attempts = 1

Stream: {"node": "retriever", "output": {...}}

STEP 4 – Evaluator Node (LLM call #2)

Input: user message + 4 chunks

LLM: Groq Llama 3.3 70B, temperature=0

Output: "SUFFICIENT\nChunks contain actionable frameworks for manager communication."

State: is_enough = True

Stream: {"node": "evaluator", "output": {"is_enough": true}}

→ (If INSUFFICIENT, would go to Query Refiner and loop back, max 3 times)

STEP 5 – Psychologist Node (LLM call #3)

Input: 4 chunks as context + user message + conversation history

LLM: Groq Llama 3.3 70B, temperature=0.7

Output: "I hear how frustrating it must feel when your ideas go unacknowledged."

According to Crucial Conversations, this situation calls for creating

psychological safety before raising concerns..."

State: final_response = "...", active_frameworks = ["Crucial Conversations", ...]

Stream: {"node": "psychologist", "output": {"final_response": "..."}}

STEP 6 – Action Planner Node (LLM call #4)

Input: psychologist's response + chunks + user message

LLM: Groq Llama 3.3 70B, temperature=0.5 + PydanticOutputParser

Output: {

steps: [

"Before your next meeting, write down 2-3 specific ideas you want to share",

"Start with 'I noticed...' rather than 'You never...' to avoid defensiveness",

"Request a brief 1:1 with your manager to discuss communication preferences"

],

framework_used: "Crucial Conversations – STATE model",

book_source: "Crucial Conversations",

time_horizon: "This week"

}

State: action_plan = [...]

Stream: {"node": "action_planner", "output": {"action_plan": [...]}}

STEP 7 – Follow-up Node (LLM call #5)

Input: user message + response + action plan

LLM: Groq Llama 3.3 70B, temperature=0.6

Output: "How long has this pattern been happening – is it recent or has it been ongoing?"

State: final_response += "\n\n" + follow_up question

Stream: {"node": "follow_up", "output": {...}}

STEP 8 – END

Stream: {"node": "__end__"}

Frontend assembles full response from streamed state updates.

Total: 5 LLM calls, ~4-6 seconds end-to-end

10. The Hybrid Retrieval System

Pure vector search has a weakness: it can miss exact keyword matches. Pure keyword search has a weakness: it misses semantic meaning.

This project combines both.

BM25 — Sparse / Keyword Search

```
Query: "manager not listening meeting"

BM25 finds: chunks containing these exact words

Weight: 0.4 (40% of final score)

Returns: 8 chunks
```

BM25 (Best Match 25) is a classic information retrieval algorithm. It scores documents based on how often the query words appear, adjusted for document length. Good at finding exact terminology, names, and specific concepts.

Dense Vector Search — Semantic Search

```
Query: "manager not listening meeting"

Embeds to: [0.23, -0.15, 0.87, 0.44, ...]

ChromaDB finds: chunks whose vectors are closest in 384-dimensional space

Weight: 0.6 (60% of final score)

Returns: 8 chunks
```

Good at finding semantically related content even with different words. "Being ignored" matches "not listening" even without shared keywords.

EnsembleRetriever — Combining Both

```
hybrid = EnsembleRetriever(
    retrievers=[bm25, dense],
    weights=[0.4, 0.6]
)

# Returns up to 16 candidates (8 from each)
```

The 0.4/0.6 weighting means semantic similarity is weighted slightly higher than exact keywords, which works well for psychology content where the same concept has many

Cohere Rerank — Neural Quality Filter

The cross-encoder reads the query and each chunk together (not separately like embeddings). This gives much deeper relevance scoring. It essentially asks: "Given this specific question, how useful is this specific passage?"

Cross-encoders are slower but more accurate than bi-encoders (embeddings). By using it only as a final filter on 16 candidates (not all 5,640), we get the quality of a cross-encoder at a fraction of the cost.

Final retrieval pipeline

